Req

Req: https://hackage.haskell.org/package/req-3.13.0

```
-- Network.HTTP.Req

req :: (
 MonadHttp m,
 HttpMethod method,
 HttpBody body,
 HttpResponse response,
 HttpBodyAllowed (AllowsBody method) (ProvidesBody
body))
 => method -> Url scheme -> body -> Proxy response ->
Option scheme -> m response
reqBr :: (
 MonadHttp m,
 HttpMethod method,
 HttpBody body,
 HttpBodyAllowed (AllowsBody method) (ProvidesBody
body))
 => method -> Url scheme -> body -> (Response BodyReader
-> m a) -> Option scheme -> m a
reqCb :: (
 MonadHttp m,
 HttpMethod method,
 HttpBody body,
 HttpResponse response,
 HttpBodyAllowed (AllowsBody method) (ProvidesBody
body))
 => method -> Url scheme -> body -> Proxy response ->
Option scheme -> (Request -> m Request) -> m response
...

class MonadIO m => MonadHttp m where
 handleHttpException :: HttpException -> m a
 getHttpConfig :: HttpConfig

data HttpConfig = HttpConfig {
 httpConfigProxy :: Maybe Proxy
 ...
}
defaultHttpConfig :: HttpConfig

data HttpException =
  VanillaHttpException HttpException |
     -- to Network.HTTP.Client.HttpException
  JsonHttpException String
      -- error parsing JSON response
isStatusCodeException :: HttpException -> Maybe
(Response ())
```

Request options

```
-- Network.HTTP.Req

data GET
data POST
data PUT
data DELETE
... etc ...
class HttpMethod a where
  type AllowsBody a :: CanHaveBody
  httpMethodName :: Proxy a -> ByteString


data Scheme = Http | Https
data Url (scheme :: Scheme)
http :: Text -> Url 'Http
https :: Text -> Url 'Https
(/~) :: ToHttpApiData a => Url scheme -> a -> Url scheme
(/:) :: Url scheme -> Text -> Url scheme
urlQ :: QuasiQuote
renderUrl :: Url scheme -> Text


type Option
data Option scheme
(=:) :: (QueryParam param, ToHttpApiData a) => Text -> a
-> param
... etc ...
```

Request body

```
-- Network.HTTP.Req

data NoReqBody = NoReqBody
data ReqBodyJson a = ReqBodyJson a
data ReqBodyFile (path :: FilePath) = ReqBodyFile path
data ReqBodyBs = ReqBodyBs ByteString
data ReqBodyLbs = ReqBodyLbs ByteString
...
class HttpRequestBody body where
  getRequestBody :: body -> RequestBody
  getRequestContent :: body -> Maybe ByteString


data CanHaveBody = CanHaveBody | NoBody
type family ProvidesBody (a :: Type) :: CanHaveBody
where
  ProvidesBody NoReqBody = 'NoBody
  ProvidesBody _ = 'CanHaveBody

type family HttpBodyAllowed (allowsBody :: CanHaveBody)
```

```
(providesBody :: CanHaveBody) :: Type where
  HttpBodyAllowed 'NoBody _ = ()
  HttpBodyAllowed 'NoBody 'CanHaveBody = TypeError
('Text "This HTTP method does not allow attaching a
request body.")
```

Response

```
-- Network.HTTP.Req

data IgnoreResponse
data JsonResponse a
data BsResponse
ignoreResponse :: Proxy IgnoreResponse
jsonResponse :: Proxy JsonResponse
bsResponse :: Proxy BsResponse
...

class HttpResponse response
  HttpResponseBody response :: Type
  toVanillaResponse :: response -> Response
(HttpResponseBody response)                -- to
Network.HTTP.Client.Response
  ...

responseBody :: HttpResponse response => response ->
HttpResponseBody response
responseStatus :: HttpResponse response => response
->  Int
responseStatusMessage :: HttpResponse response =>
response ->  String
responseHeader :: HttpResponse response => response
->  ByteString -> Maybe ByteString
responseCookieJar :: HttpResponse response => response
->  CookieJar
```

```
instance HttpMethod GET where

  type AllowsBody = 'NoBody
  -- No request body
instance HttpMethod POST where
  type AllowsBody = 'CanHaveBody
 -- May have request body
instance HttpMethod PUT where
  type AllowsBody = 'CanHaveBody

instance HttpResponse (JsonResponse a) where
  type HttpResponseBody (JsonResponse a) = a
instance HttpResponse BsResponse where
  type HttpResponseBody BsResponse = ByteString
          -- strict i.e. Data.ByteString
```

```
instance HttpResponse (JsonResponse a) where
  type HttpResponseBody LbsResponse = ByteString
         -- lazy i.e. Data.ByteString.Lazy
```

e.g. call

```
{#- LANGUAGE QuasiQuotes #-}
import Network.HTTP.Req

test :: IO ()
test = do
  resp <- runReq defaultHttpConfig $ req GET (https
"www.google.com") NoReqBody bsResponse mempty
  putStrLn $ "body:" <> responseBody resp

  let (url, opts) = [urlQ|https://www.google.com:443|]
  resp <- runReq defaultHttpConfig $ req GET url
NoReqBody bsResponse opts
  putStrLn $ "body:" <> responseBody resp
```

```
-- Basics
--
import Data.ByteString (ByteString)

req GET (http "localhost" /: "a" /: "b") NoReqBody
ignoreResponse mempty
req POST ...
req PUT ...
req DELETE ...

-- request body
req GET (http "localhost") NoReqBody ....
req GET (http "localhost") (ReqBodyJson [1 ::
Int,2,3]) ...          -- Content-Type:     application/
json "[1,2,3]"
req GET (http "localhost") (ReqBodyFile "./data/
text") ...
req GET (http "localhost") (ReqBodyBs "Hello") ....
req GET (http "localhost") (ReqBodyLbs "Hello") ....

-- response format
req GET (http "localhost") NoReqBody ignoreResponse ...
req GET (http "localhost") NoReqBody (jsonResponse
@String)...
req GET (http "localhost") NoReqBody bsResponse ...
req GET (http "localhost") NoReqBody lbsResponse ...

-- request options
req ... ignoreResponse mempty
req ... ignoreResponse (port 1000 <> header "A" "1")
req ... ignoreResponse (port 1000)
```

```
req ... ignoreResponse (header "A" "1" <> header "B"
"2")        -- Headers  A:1 B:2
req ... ignoreResponse ("a" =: (1 :: Int) <> "b" =:
True)        -- ?a=1&b=true
... etc

---- URL
--
-- with quasi quotes
{#- LANGUAGE QuasiQuotes -#}
[urlQ|https://localhost:80/a/b?v1=1] :: (Url 'Https,
Options 'Https)        --
(
  (https "localhost") /: "a" /:
"b",                                          -- same
  (port 80) <> v1 =: (1 :: Int)
)

-- response
response <- runReq defaultHttpConfig $ req GET ...
(jsonResponse @[Int])
responseBody response ::
[Int]                                  -- e.g. [1,2,3]
responseStatusCode response ::
Int                             -- e.g. 200
toVanillaResponse response :: Response (HttpResponseBody
```